



Using a variable neighborhood search to solve a bi-objective identical parallel machine scheduling problem

Leandro Resende Mundim^{a,1} Thiago Alves de Queiroz^{b,2}

^a *Institute of Mathematical and Computer Sciences, University of São Paulo, Av. Trabalhador São-carlense, 400, 13566-590, São Carlos - SP, Brazil*

^b *Institute of Mathematics and Technology, Federal University of Goiás - Catalão, Av. Dr. Lamartine P. de Avelar, 1120, 75704-020, Catalão - GO, Brazil*

Abstract

We developed a variable neighborhood search heuristic and a mixed integer programming model for the identical parallel machine scheduling problem with sequence dependent setup time. For this problem, we consider minimizing two objectives, which are the makespan and the flow time. The heuristic proposed has a constructive procedure to build initial solutions, five neighborhood structures, and a local search based on the variable neighborhood descent. Computational experiments indicate that the heuristic is very fast and can return better solutions than the model since it found 90% of the best solutions. It also outperformed all solutions computed with the longest processing time and the shortest processing time rules, both commonly adopted for scheduling problems.

Keywords: variable neighborhood search, parallel machines, makespan, flow time.

¹ Email: mundim@icmc.usp.br

² Email: taq@ufg.br

1 Introduction

Determining the best sequence to schedule n jobs, each job with a given processing time and sequence dependent setup time, on m identical parallel machines, is a practical problem which has applications in manufacturing, air traffic control, project management, among others. This problem is NP-hard if we are interested in minimizing the total time when scheduling all jobs [11]. An extensive survey of scheduling problems involving setup times and costs was presented in [1].

In the three-field notation, the identical parallel machine scheduling problem is denoted as $Pm|S_{ij}|C_{max}$ if minimizing the makespan (the completion time of the last job). On the other hand, it is denoted as $Pm|S_{ij}|\sum C_i$ if minimizing the total flow time (the sum of the completion time of all jobs). In this paper, we consider both objectives simultaneously, such that the notation $Pm|S_{ij}|C_{max}, \sum C_i$ is adopted. By considering these two objectives together, there is a reduction in the time in which the shop should operate, in addition to making better use of available resources, then there will be a reduction in the production costs [9].

Regarding the minimization of the flow time, Gupta and Ho [6] proposed an optimization algorithm considering two identical machines when knowing the optimal makespan value in advance. Under a hierarchical approach, Gupta and Ruiz-Torres [5] considered the minimization of the makespan but with the optimal flow time known for which they proposed a heuristic combining a list-scheduling procedure with a backtracking algorithm. A multi start simulated annealing algorithm for a permutation flow shop problem with the minimization of the makespan and the total flow time was proposed in [9]. It is important to mention that there are few studies involving parallel machine problems solved with variable neighborhood search based heuristics [2,3,4].

2 Mathematical Model

A mixed integer programming (MIP) model for the $Pm|S_{ij}|C_{max}, \sum C_i$ is presented next. We consider the minimization of the makespan and the flow time together in which a factor α is used to weigh one in relation to the other, forming a linear combination. The parameters used in the model are: J , the set of $n + 1$ jobs in which “0” is a fictitious job; M , the set of m identical machines; p_i , the processing time of job i , for all $i \in J$, except for $p_0 = 0$; and, s_{ij} , the setup time when processing job j immediately after job i , for all $i, j \in J$. If $i = j$ or conversely $i \neq j$ but $i = 0$ ($j = 0$), then the setup

time is the time required for setup machine $k \in M$ given j (i) as the first job scheduled on k .

The variables used in the model are: x_{ijk} , which is 1 if job i precedes immediately job j on machine k , and 0 otherwise, for all $i, j \in J$ and $k \in M$, where $x_{0ik} = 1$ means that i is the first job scheduled on machine k and $x_{i0k} = 1$ means that i is the last job scheduled on machine k ; $C_i \geq 0$, the completion time of job i , for all $i \in J$; and, $C_{max} \geq 0$, the makespan.

$$\text{Minimize} \quad (1 - \alpha)C_{max} + \alpha \sum_{i \in J} C_i \quad (1)$$

Subject to:

$$\sum_{i \in J} \sum_{k \in M} x_{ijk} = 1, \quad \forall j \in J, \quad (2)$$

$$\sum_{i \in J} x_{ihk} = \sum_{j \in J} x_{hjk}, \quad \forall h \in J, \forall k \in M, \quad (3)$$

$$\sum_{j \in J} x_{0jk} = 1, \quad \forall k \in M, \quad (4)$$

$$x_{0ik}(s_{ii} + p_i) \leq C_i, \quad \forall i \in J, \forall k \in M, \quad (5)$$

$$C_j \geq C_i + s_{ij} + p_j + V\left(\sum_{k \in M} x_{ijk} - 1\right), \quad \forall i, j \in J, \quad (6)$$

$$C_i \leq C_{max}, \quad \forall i \in J, \quad (7)$$

$$C_i \geq 0, \quad \forall i \in J, \quad (8)$$

$$x_{ijk} \in \{0, 1\}, \quad \forall i, j \in J, \forall k \in M. \quad (9)$$

The objective function (1) is related to minimizing the linear combination of the two objectives, makespan and flow time, for a given weight factor $\alpha \in [0, 1]$. Constraints (2) ensure that each job is scheduled exactly once. Constraints (3) guarantee the flow on each machine. Constraints (4) impose that each machine has only one job scheduled as the first job. Constraints (5) ensure the completion time for the first job i scheduled on each machine k .

In order to avoid sub-tours, as well as estimate the completion time for each job i , we consider the well-known Miller-Tucker-Zemlin (MTZ) elimination constraints in (6) considering a big value V . Constraints (7) give the value of the makespan for C_{max} . Finally, constraints (8) and (9) impose the domain of the variables.

3 Variable Neighborhood Search Heuristic

The Variable Neighborhood Search (VNS) is able to explore different neighborhoods systematically [10]. We consider the VNS in Algorithm 1 in which its local search phase is the sequential Variable Neighborhood Descent described in Algorithm 2.

Algorithm 1: VNS FOR THE $Pm|S_{ij}|C_{max}, \sum C_i$.

Input: J, p_i, s_{ij}, α , and M .

Output: s^* is the solution with the best objective value.

```

1  $s^* \leftarrow$  initial solution by a constructive heuristic.
2 for  $g \leftarrow 1$  to maximum number of iterations do
3    $k \leftarrow 1$ ;
4   while  $k \leq$  number of neighborhood structures do
5      $s' \leftarrow$  random neighbor from  $s^*$  in neighborhood  $N_k$ ;
6      $s'' \leftarrow$  Local-Search( $s'$ );
7     if  $f(s'', \alpha) < f(s^*, \alpha)$  then
8        $s^* \leftarrow s''$ ;
9        $k = 1$ ;
10    else
11       $k++$ ;
12    end
13  end
14 end
15 return  $s^*$ ;

```

Algorithm 1 requires an initial solution, which is obtained by a constructive heuristic. We propose a biased version of the longest processing time rule (B-LPT) for scheduling all the jobs, as being the constructive heuristic. First, it calculates the average setup time observing all the setup times and then it sorts all the jobs in a non-increasing order observing an artificial processing time for i as $\beta \times (\text{processing time of } i + \text{average setup time})$, for a biased $\beta \in [0.8, 1]$ for each job. After sorting, it schedules the jobs in such an order by using the longest processing time (LPT) rule [11].

An illustrative example of the constructive heuristic is given in Figure 1. We consider 05 jobs and 03 machines in which the jobs have been sorted

Algorithm 2: SEQ-VND AS LOCAL-SEARCH**Input:** Solution s and value of α .**Output:** Solution s possible improved.

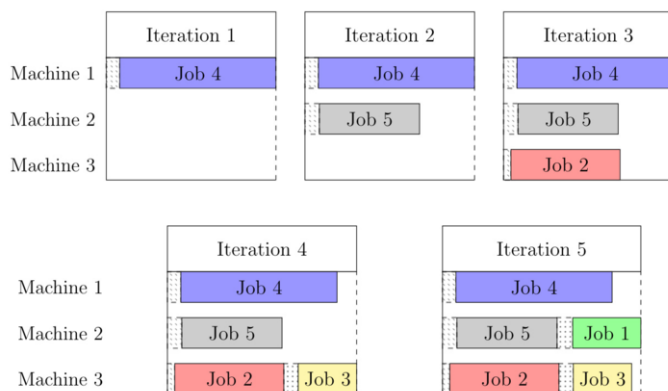
```

1  $k \leftarrow 1$ ;
2 while  $k \leq \text{number of neighborhood structures}$  do
3   while  $s$  is not a local optimal solution for neighborhood  $N_k$  do
4      $s'$  is the first solution in  $N_k$  better than  $s$  for a given  $\alpha$ ;
5      $s \leftarrow s'$ ;
6   end
7    $k++$ ;
8 end
9 return  $s$ ;

```

observing their artificial processing time, then resulting in the order: 04, 05, 02, 03, and 01. Note that jobs are not allocated exactly at the beginning of any machine and not even at the end of their immediately previous jobs because the setup time is sequence dependent.

Fig. 1. Example of how an initial solution is constructed.



The performance of the VNS algorithm is intrinsically related with its neighborhood structures. We consider five types of structures that together have allowed the VNS to obtain the best solutions for almost all the instances. The structures are:

- Neighborhood N_1 : (a) move the shortest job, in terms of processing time, of the machine with the longest makespan to the beginning of the machine with the shortest makespan; and, (b) move the longest job of the machine with the longest flow time to the end of the machine with the shortest flow time.

- Neighborhood N_2 : (a) for the machines with the longest and the shortest makespan, swap their jobs in order to balance their makespan; and, (b) apply the shortest processing time (SPT) rule in all machines individually [11], aiming to reduce the flow time.
- Neighborhood N_3 : (a) swap two jobs chosen at random from two machines also randomly chosen; (b) move three jobs chosen at random from three machines also randomly chosen, where the first job is moved to the position of the second job, the second job goes to the position of the third job, and the third job is moved to the position of the first job; and (c) swap two jobs chosen at random from one machine also randomly chosen. After swapping, the shortest job between those chosen is positioned immediately before the job that precedes it.
- Neighborhood N_4 : apply the Lin-Kernighan heuristic [8], which is based on λ -opt moves, for swapping jobs on each machine.
- Neighborhood N_5 : remove randomly a subset of jobs observing all machines and schedule this subset by applying the constructive heuristic on it.

4 Computational Experiments

All the algorithms were implemented in the C++ programming language and the computational experiments were performed on a machine with Intel® Core™ i7-2600, 3.40 GHz, 16 GB RAM and Ubuntu 12.04 as the operating system. The MIP model was solved with the IBM ILOG CPLEX 12.7 considering its default settings.

The experiments presented next have been strongly based on the ones in [7], where the authors compared their methods with the LPT and SPT rules, as well as proposed their own instances. In Table 1, we present the results for the 30 instances generated in accordance with [7]. Each line of the table has: instance name; quantity of machines (m); quantity of jobs (n); results of the VNS, which are the best solution (S^*), average solution ($\bar{S}$), and average runtime in seconds ($\overline{T(s)}$), for 10 runs and time limit of 120 seconds per run, and $\alpha = 0.7$; results of the LPT and SPT rules; and, results of the MIP model, which are the solution (SOL) and the runtime in seconds ($\overline{T(s)}$), for a time limit (TL) of 3600 seconds.

The results in Table 1 clearly show that the VNS outperformed the LPT and SPT rules. In comparison with the MIP model, the VNS improved almost all the non-optimal solutions, as well as obtaining all optimal solutions. The VNS was worse than the MIP model only for the instances 9, 10, and 14.

Table 1
Results for the set of 30 randomly generated instances.

Ins	m	n	VNS			LPT	SPT	MIP model	
			S*	$\overline{S}$	$\overline{T(s)}$	SOL	SOL	SOL	T(s)
1	2	6	397.1	397.1	5	403.9	464.2	397.1	0
2	2	8	601.5	601.5	6	636.9	704.8	601.5	10
3	2	10	657.0	657.0	7	734.1	869.8	657.0	628
4	2	14	1301.7	1309.7	11	1407.3	1822.1	1301.7	TL
5	2	20	2124.0	2137.4	16	2436.3	3071.0	2126.8	TL
6	3	9	466.1	466.1	27	494.5	553.1	466.1	22
7	3	12	716.2	716.2	96	818.4	973.4	716.2	TL
8	3	15	910.4	919.9	120	984.4	1237.3	910.4	TL
9	3	21	1744.9	1758.0	120	1946.6	2441.9	1730.0	TL
10	3	30	2906.3	2959.3	120	3227.4	4284.9	2893.6	TL
11	4	12	582.6	582.6	48	626.6	796.3	582.6	1885
12	4	16	803.9	809.4	115	901.6	1068.7	803.9	TL
13	4	20	1188.0	1200.7	120	1320.9	1680.2	1193.4	TL
14	4	28	2167.6	2183.9	120	2373.2	3086.3	2160.9	TL
15	4	40	4226.3	4264.8	120	4543.7	6319.2	4267.1	TL
16	5	15	617.2	617.4	72	669.4	821.4	617.2	TL
17	5	20	1000.8	1012.7	120	1128.0	1349.4	1011.4	TL
18	5	25	1607.7	1621.6	120	1711.7	2318.2	1614.6	TL
19	5	35	2808.5	2821.1	120	2992.3	3891.2	2966.5	TL
20	5	50	4479.4	4538.3	120	4913.6	6723.0	4705.3	TL
21	7	21	897.5	904.4	93	978.3	1154.9	899.6	TL
22	7	28	1377.8	1392.2	120	1514.7	1894.1	1427.3	TL
23	7	35	2122.5	2137.4	120	2278.2	2869.8	2241.6	TL
24	7	49	3372.9	3399.6	120	3743.6	5056.7	3601.5	TL
25	7	70	6189.4	6295.2	120	6517.4	9311.9	6633.5	TL
26	10	30	1124.4	1132.3	120	1216.9	1499.4	1202.8	TL
27	10	40	1947.3	1965.8	120	2163.9	2781.4	2073.7	TL
28	10	50	2445.0	2480.5	120	2744.2	3611.8	2767.1	TL
29	10	70	4494.0	4583.8	120	4888.2	6679.0	5190.7	TL
30	10	100	9842.0	9930.1	120	10427.7	13816.4	13372.3	TL

5 Concluding Remarks

The variable neighborhood search developed for the identical parallel machine scheduling problem with sequence dependent setup time was able to return 90% of the best solutions. Moreover, it improved results of well-defined rules for scheduling problems, namely the LPT and SPT rules, as well as those of the MIP model. The average difference between the solutions of the VNS with the LPT, SPT and the MIP model is of 9.13%, 38.24%, and 3.78%, respectively, for the set of 30 instances. Future studies could consider different values for the weight factor α in order to construct a Pareto curve and a more in-depth evaluation of the neighborhood structures adopted.

Acknowledgement

The authors would like to thank CAPES, CNPq (grant 308312/2016-3) and PPGMO for their financial support.

References

- [1] Allahverdi, A., *The third comprehensive survey on scheduling problems with setup times/costs*, European Journal of Operational Research **246** (2015), pp. 345–378.
- [2] Bathrinath, S., S. Saravana Sankar, S. G. Ponnambalam and I. Jerin Leno, *Vns-based heuristic for identical parallel machine scheduling problem*, in: L. P. Suresh, S. S. Dash and B. K. Panigrahi, editors, *Artificial Intelligence and Evolutionary Algorithms in Engineering Systems: Proceedings of ICAEES 2014, Volume 1*, Springer India, New Delhi, 2015 pp. 693–699.
- [3] Behnamian, J., M. Zandieh and S. F. Ghomi, *Parallel-machine scheduling problems with sequence-dependent setup times using an ACO, SA and VNS hybrid algorithm*, Expert Systems with Applications **36** (2009), pp. 9637–9644.
- [4] de Paula, M. R., M. G. Ravetti, G. R. Mateus and P. M. Pardalos, *Solving parallel machines scheduling problems with sequence-dependent setup times using variable neighbourhood search*, IMA Journal of Management Mathematics **18** (2007), p. 101.
- [5] Gupta, J. N. and A. J. Ruiz-Torres, *Minimizing makespan subject to minimum total flow-time on identical parallel machines*, European Journal of Operational Research **125** (2000), pp. 370–380.
- [6] Gupta, J. N. D. and J. C. Ho, *Minimizing flowtime subject to optimal makespan on two identical parallel machines*, Pesquisa Operacional **20** (2000), pp. 05–17.
- [7] Hamzadayi, A. and G. Yildiz, *Modeling and solving static m identical parallel machines scheduling problem with a common server and sequence dependent setup times*, Computers & Industrial Engineering **106** (2017), pp. 287–298.
- [8] Lin, S. and B. W. Kernighan, *An effective heuristic algorithm for the traveling-salesman problem*, Operations Research **21** (1973), pp. 498–516.
- [9] Lin, S.-W. and K.-C. Ying, *Minimizing makespan and total flowtime in permutation flowshops by a bi-objective multi-start simulated-annealing algorithm*, Computers & Operations Research **40** (2013), pp. 1625–1647.
- [10] Mladenovi, N. and P. Hansen, *Variable neighborhood search*, Computers & Operations Research **24** (1997), pp. 1097–1100.
- [11] Pinedo, M. L., “Scheduling: Theory, Algorithms, and Systems,” Springer Publishing Company, Incorporated, 2008, 3rd edition.